



Security-Centric Architecture Assessment Framework

Complete Project Documentation

B.Tech Major Project — Computer Science & Engineering

Domain: Cybersecurity & Software Architecture

Methodology: STRIDE Threat Modeling | Security by Design

Version 1.0.0 | Academic Year 2025–2026

Table of Contents

No.	Section
1	Project Overview & Abstract
2	Problem Statement
3	Existing Tools & Research Gap
4	Proposed System
5	System Architecture
6	Technology Stack
7	Core Modules Explained
8	STRIDE Threat Modeling
9	DREAD Risk Scoring
10	OWASP Top 10 Integration
11	Architecture Patterns (12)
12	Security Pattern Library (24)
13	Attack Tree Visualization
14	Secure Coding Examples
15	AI Security Assistant
16	User Authentication System
17	Report Generation
18	Sample Architecture Files
19	Functional Requirements
20	Non-Functional Requirements
21	Test Cases
22	Future Scope
23	Conclusion

1. Project Overview & Abstract

The Security-Centric Architecture Assessment Framework is a web-based automated security analysis platform developed as a B.Tech Major Project. It implements the principle of **Security by Design** — analyzing software architecture descriptions for security vulnerabilities before any code is written.

Traditional security tools such as OWASP ZAP, Burp Suite, and SonarQube require running applications or source code to perform analysis. This framework uniquely operates at the **architecture design stage**, identifying security threats when they are least expensive to fix — during design, not after deployment.

Key Capabilities:

- Automated STRIDE threat modeling across 6 categories
- DREAD quantitative risk scoring
- OWASP Top 10 (2021) threat mapping
- 12 detailed security-centric architecture patterns
- 24 curated security design patterns with implementation guidance
- Interactive attack tree visualization
- AI-powered security architecture assistant (Groq LLaMA 3)
- Secure coding examples — vulnerable vs. fixed code
- Professional HTML, PDF, JSON, and CSV report generation
- Multi-format input: JSON, YAML, XML, PlantUML
- SQLite-based analysis history with reload capability
- Role-based user authentication (Admin/Faculty/Student)

2. Problem Statement

Software security vulnerabilities are most commonly discovered during the testing or production phases of development. Industry research consistently shows that a security flaw fixed at the design stage costs 10 to 100 times less to remediate than the same flaw discovered after deployment. Despite this, most development teams lack accessible tools for automated security analysis at the architecture level.

Identified Problems:

ID	Problem
P1	Security vulnerabilities discovered late — after coding, testing, or deployment
P2	Late-discovered vulnerabilities require expensive architectural redesign
P3	Developers lack automated tools for architecture-level security assessment
P4	No integrated workflow combining STRIDE, DREAD, OWASP, patterns, and reporting
P5	Security patterns not systematically applied during the design phase
P6	Manual threat modeling requires significant time and security expertise

3. Existing Tools & Research Gap

Tool	Stage	Requires Code	Architecture-Level	Automated STRIDE	Reports
OWASP ZAP	Post-deploy	Yes	No	No	Yes
Burp Suite	Testing	Yes	No	No	Yes
Nessus	Post-deploy	Yes	No	No	Yes
SonarQube	Development	Yes	No	No	Yes
Checkmarx	Development	Yes	No	No	Yes
OWASP Threat Dragon	Design	No	Yes	No	No
MS Threat Model Tool	Design	No	Yes	No	Limited
THIS FRAMEWORK	Design	No	Yes	Yes	Yes

The research gap clearly shows that no existing tool provides an integrated automated workflow combining STRIDE analysis, DREAD scoring, OWASP mapping, security pattern recommendations, and professional report generation — all operating on architecture descriptions without requiring code or deployment.

4. Proposed System

The proposed system is a full-stack web application built with Python and Streamlit that accepts architecture descriptions in multiple formats and performs comprehensive automated security analysis. The system consists of 17 functional pages and 8 core backend modules working together in a pipeline.

System Pipeline:

Step	Phase	Description
Step 1	Input	User uploads JSON/YAML/XML/PlantUML architecture file
Step 2	Parsing	Format-specific parser normalizes input to common data model
Step 3	Validation	Architecture validator checks structure and references
Step 4	STRIDE Analysis	20+ rules applied to every component and data flow
Step 5	Risk Calculation	Likelihood x Impact scoring with DREAD enhancement
Step 6	OWASP Mapping	Each threat mapped to OWASP Top 10 2021 categories
Step 7	Pattern Mapping	Threats mapped to 24 security design patterns
Step 8	Recommendations	Prioritized, architecture-specific fix list generated
Step 9	Visualization	Interactive graph + heatmap + threat overlay
Step 10	Report	HTML, PDF, JSON, CSV reports generated for download

5. System Architecture

Module Structure:

File / Module	Responsibility
app.py	Main Streamlit entry point, navigation router, sidebar
config.py	Application configuration, student details, risk thresholds
core/models.py	Data models: Architecture, Component, DataFlow, Threat, AnalysisResult
core/analyzer.py	Main orchestrator — connects all 4 engines
core/validators.py	Architecture validation and safe type conversion
core/exceptions.py	Custom exception hierarchy
parsers/json_parser.py	JSON architecture file parser
parsers/yaml_parser.py	YAML architecture file parser
parsers/xml_parser.py	XML architecture file parser
parsers/plantuml_parser.py	PlantUML diagram parser
threat_engine/stride_engine.py	STRIDE rule application engine with deduplication
threat_engine/threat_rules.py	20+ STRIDE detection rules for components and flows
risk_engine/risk_calculator.py	Likelihood x Impact risk scoring formula
patterns/security_patterns.py	24 curated security design patterns library
patterns/threat_pattern_mapper.py	Maps threats to security patterns
recommendations/recommendation_engine.py	Generates prioritized remediation recommendations
visualization/architecture_graph.py	NetworkX + Plotly interactive graph builder
visualization/dashboard_charts.py	9 Plotly chart generators
reports/html_generator.py	Dark-themed HTML report generator
reports/pdf_generator.py	ReportLab PDF report generator
reports/report_generator.py	Report orchestrator — JSON and CSV exports
database/db.py	SQLite analysis history — parameterized queries
utils/session_manager.py	Streamlit session state management
utils/logger.py	Application logging setup
utils/helpers.py	File validation and sanitization utilities
page_modules/ (17 files)	Individual page modules — each exports show()

6. Technology Stack

Technology	Version	Purpose	Why Chosen
Python	3.12	Primary language	Dominant in ML/security tooling, rapid development
Streamlit	1.35.0	Web framework	Fastest Python UI framework, no HTML/JS needed
Plotly	5.22.0	Interactive charts	Best Python library for interactive visualizations
NetworkX	3.3	Graph construction	Industry-standard graph algorithms library
Pandas	2.2.2	Data tables & CSV	Standard data manipulation library
PyYAML	6.0.1	YAML parsing	Safe YAML parsing with security defaults
ReportLab	4.2.0	PDF generation	Professional PDF creation without external tools
Jinja2	3.1.4	HTML templating	Secure HTML generation with auto-escaping
SQLite3	Built-in	Analysis history DB	Zero-dependency embedded database
Groq	1.5.0	AI chat (LLaMA 3)	Free tier, fastest LLM inference available
bcrypt	4.0.1	Password hashing	Industry standard slow hash for passwords
hashlib	Built-in	User auth hashing	SHA-256 with salt for local auth system
pytest	8.2.2	Automated testing	Standard Python testing framework

7. Core Modules Explained

7.1 Data Models (core/models.py)

All architecture parsers convert their input into standardized Python dataclass objects. This normalization ensures that JSON, YAML, XML, and PlantUML files all produce identical data structures for the analysis engine.

Class	Fields	Purpose
Component	id, name, type, zone, internet_facing, data_sensitivity, authentication, logging_enabled, r	Represents a system component or database, API endpoint, etc.
DataFlow	id, source, destination, protocol, data, encrypted, authentication, r	Represents a data flow between two components
TrustBoundary	id, name, zone_from, zone_to, description	Represents a security zone crossing boundary
Architecture	id, name, description, components, data_flows, trust_boundaries, r	Top-level container for entire architecture
Threat	id, stride_category, title, severity, risk_score, likelihood, impact, r	One detected security threat
AnalysisResult	architecture, threats, risk_summary, pattern_mappings, recommendations	Complete output of the analysis pipeline

7.2 Parsers (parsers/)

Four format-specific parsers all inherit from BaseParser and implement a single parse(content: str) -> Architecture method. Each parser handles format-specific syntax and maps it to the common Architecture data model.

Parser	Input Format	Key Implementation Detail
JSONParser	JSON (.json)	Uses json.loads() with safe_str/safe_bool helpers for missing fields
YAMLParse	YAML (.yaml/.yml)	Uses yaml.safe_load() — delegates to JSONParser._build_architecture()
XMLParser	XML (.xml)	Uses xml.etree.ElementTree — handles both attribute and nested formats
PlantUMLParser	PlantUML (.puml/.txt)	Regex-based parser — extracts components from keywords, arrows from -> patterns

8. STRIDE Threat Modeling

STRIDE is a threat modeling methodology developed by Microsoft. It provides a systematic framework for identifying security threats by categorizing them into 6 types. This framework implements STRIDE using rule-based detection applied to every component and data flow in the architecture.

Category	Meaning	What It Checks	Mitigation Pattern
Spoofing	Impersonating identity	authentication=False on internet-facing components	Strong Authentication, MFA, JWT
Tampering	Modifying data/code	input_validation=False, encrypted=False on flows	Input Validation, Encryption in Transit
Repudiation	Denying performed actions	logging_enabled=False on sensitive components	Secure Logging, Audit Trail
Information Disclosure	Leaking confidential data	encryption_at_rest=False on sensitive databases	Encryption at Rest, Network Segmentation
Denial of Service	Disrupting availability	rate_limiting=False on internet-facing services	Rate Limiting, Circuit Breaker
Elevation of Privilege	Gaining unauthorized access	authorization=False on internal services	RBAC, Least Privilege, Zero Trust

Detection Rules (20+ total):

Rule ID	Category	Trigger Condition	Severity
SPO-001	Spoofing	internet_facing=True AND authentication=False	Critical
SPO-002	Spoofing	type=external AND authentication=False	High
SPO-003	Spoofing	'admin' in name AND authentication=False	Critical
TAM-001	Tampering	input_validation=False AND (internet_facing OR is_api)	High
TAM-002	Tampering	type=database AND authorization=False	High
TAM-003	Tampering	data_sensitivity=high/critical AND encryption=False	High
REP-001	Repudiation	logging_enabled=False AND not external	Medium
REP-002	Repudiation	data_sensitivity=high AND logging_enabled=False	High
INF-001	InformationDisclosure	type=database AND internet_facing=True	Critical
INF-002	InformationDisclosure	is_db AND sensitive AND encryption_at_rest=False	Critical
INF-003	InformationDisclosure	is_api AND logging=False AND encryption=False	High
DOS-001	DenialOfService	internet_facing=True AND rate_limiting=False	High
EOP-001	ElevationOfPrivilege	authorization=False AND not external	High
EOP-002	ElevationOfPrivilege	is_admin AND authorization=False	Critical
EOP-003	ElevationOfPrivilege	type=service AND authorization=False	Medium
TAM-F001	Tampering (Flow)	flow.encrypted=False	High
TAM-F002	Tampering (Flow)	flow.protocol in HTTP/FTP/TELNET	High
SPO-F001	Spoofing (Flow)	flow.authenticated=False	Medium
INF-F001	InformationDisclosure	flow unencrypted AND carries sensitive data keywords	Critical

EOP-F001	ElevationOfPrivilege	flow crosses boundary AND unauthenticated	High
----------	----------------------	---	------

9. DREAD Risk Scoring

DREAD is a quantitative risk scoring methodology developed by Microsoft that assigns numeric values to five factors, producing an overall risk score for each threat. This framework implements both the STRIDE detection (for finding threats) and DREAD scoring (for quantifying their risk).

Factor	Letter	Question Asked	Score Range
Damage Potential	D	How bad is the damage if successfully exploited?	0–10
Reproducibility	R	How easy is it to reproduce the attack?	0–10
Exploitability	E	How easy is it to launch the attack?	0–10
Affected Users	A	How many users are affected if exploited?	0–10
Discoverability	D	How easy is it to discover this vulnerability?	0–10

DREAD Score = (D + R + E + A + D) / 5

Classification: 0–3 = Low Risk | 4–6 = Medium Risk | 7–9 = High Risk | 10 = Critical

The framework also implements the primary risk formula: Risk Score = Likelihood (1–5) x Impact (1–5). Classification: 1–4 = Low | 5–9 = Medium | 10–16 = High | 17–25 = Critical. Overall Architecture Risk % = (Average Score / 25) x 100.

10. OWASP Top 10 Integration

The Open Web Application Security Project (OWASP) Top 10 is a globally recognized standard for web application security risks. This framework maps each STRIDE threat to the corresponding OWASP Top 10 2021 category.

OWASP ID	Vulnerability	Severity	STRIDE Mapping
A01:2021	Broken Access Control	Critical	Spoofing, ElevationOfPrivilege
A02:2021	Cryptographic Failures	Critical	InformationDisclosure, Tampering
A03:2021	Injection	Critical	Tampering
A04:2021	Insecure Design	High	All STRIDE categories
A05:2021	Security Misconfiguration	High	InformationDisclosure, DoS
A06:2021	Vulnerable and Outdated Components	High	Tampering, InformationDisclosure
A07:2021	Identification and Auth Failures	High	Spoofing
A08:2021	Software and Data Integrity Failures	Medium	Tampering, Repudiation
A09:2021	Security Logging and Monitoring Fail	Medium	Repudiation
A10:2021	Server-Side Request Forgery (SSRF)	Medium	InformationDisclosure, DoS

11. Architecture Patterns (12 Patterns)

The framework includes 12 detailed security-centric architectural patterns. Unlike security patterns (which fix individual controls), architecture patterns define how the entire system should be structured for security.

ID	Pattern Name	Category	Primary STRIDE Addressed	Best For
AP-001	Layered Architecture	Structural	Tampering, ElevationOfPrivilege	Web apps, REST APIs
AP-002	Microservices Security	Distributed	Spoofing, InformationDisclosure	Large-scale SaaS
AP-003	Zero Trust Architecture	Security	All 6 STRIDE categories	Cloud, remote workforce
AP-004	Secure MVC Pattern	Structural	Tampering, Spoofing	Django, Spring, Rails
AP-005	Event-Driven Secure Arch	Distributed	Tampering, Repudiation	Banking, IoT, fraud detection
AP-006	Defense in Depth	Security	All 6 STRIDE categories	Banking, healthcare, govt
AP-007	Secure API Gateway	Infrastructure	DoS, Spoofing, EoP	Public APIs, mobile backends
AP-008	Broker Security Pattern	Distributed	Spoofing, Tampering	Enterprise middleware
AP-009	Service Mesh Security	Infrastructure	Spoofing, InformationDisclosure	Kubernetes, cloud-native
AP-010	Secure Client-Server	Structural	Tampering, InformationDisclosure	All web applications
AP-011	CQRS Security Pattern	Structural	Tampering, Repudiation	Financial, audit-heavy systems
AP-012	Hexagonal Architecture	Structural	All via adapter boundaries	Complex enterprise, DDD

12. Security Pattern Library (24 Patterns)

ID	Pattern	Category	Priority	STRIDE
SP-001	Strong Authentication	Authentication	Critical	Spoofing
SP-002	Multi-Factor Authentication	Authentication	Critical	Spoofing
SP-003	Role-Based Access Control	Authorization	High	ElevationOfPrivilege, Tampering
SP-004	Attribute-Based Access Control	Authorization	Medium	ElevationOfPrivilege
SP-005	Least Privilege	Authorization	High	ElevationOfPrivilege
SP-006	Encryption in Transit	Cryptography	Critical	InformationDisclosure, Tampering
SP-007	Encryption at Rest	Cryptography	High	InformationDisclosure, Tampering
SP-008	Input Validation	Input Handling	Critical	Tampering, ElevationOfPrivilege
SP-009	Output Encoding	Input Handling	High	Tampering, InformationDisclosure
SP-010	Secure Session Management	Authentication	High	Spoofing
SP-011	API Gateway	Infrastructure	High	DoS, Spoofing, EoP
SP-012	Rate Limiting	Availability	High	DenialOfService
SP-013	Secure Logging	Monitoring	High	Repudiation
SP-014	Audit Trail	Monitoring	High	Repudiation
SP-015	Integrity Verification	Cryptography	Medium	Tampering
SP-016	Digital Signatures	Cryptography	Medium	Repudiation, Tampering
SP-017	Secrets Management	Configuration	High	InformationDisclosure
SP-018	Network Segmentation	Infrastructure	High	InformationDisclosure, EoP
SP-019	Circuit Breaker	Availability	Medium	DenialOfService
SP-020	Redundancy and Failover	Availability	High	DenialOfService
SP-021	Security Headers	Web Security	Medium	Tampering, InformationDisclosure
SP-022	Zero Trust	Architecture	High	Spoofing, EoP, InformationDisclosure
SP-023	Token-Based Authentication	Authentication	High	Spoofing
SP-024	Database Access Control	Data Security	High	Tampering, InformationDisclosure, EoP

13. Attack Tree Visualization

Attack trees are a formal method for analyzing the security of systems. They represent attacks against a system in a tree structure where the root is the attacker's goal and leaf nodes are specific attack methods. This framework includes both pre-built and dynamically generated attack trees.

Pre-Built Attack Trees:

Tree	Root Goal	Attack Vectors	Leaf Attacks
Unauthorized Data Access	Gain unauthorized access to sensitive data	Authentication weakness, Authz flaw, DB exploit	Brute force, phishing, SQL injection, IDOR
API Service Disruption	Disrupt API service availability	Volumetric, resource exhaustion, logic flaw	DDoS, HTTP flood, Slowloris, payload overflow
Identity Spoofing	Impersonate legitimate user/service	Credential theft, token forgery, MITM	Phishing, JWT alg=none, replay attack
Data Tampering	Modify data without authorization	Injection, traffic intercept, insider	SQL injection, XSS, MITM, compromised accounts

Gate Types: AND gate = all child conditions must be true for the attack to succeed. OR gate = any single child condition is sufficient. Node color indicates attack probability: Red = high (>60%), Yellow = medium (40-60%), Green = low (<40%).

14. Secure Coding Examples

The Secure Coding module provides 6 side-by-side examples showing vulnerable code, the attack scenario, and the fixed production-ready code.

ID	Vulnerability	OWASP	Severity	Key Fix
SC-001	SQL Injection	A03:2021	Critical	Parameterized queries — never string concatenation
SC-002	Cross-Site Scripting	A03:2021	High	Output encoding + Content Security Policy headers
SC-003	Hardcoded Secrets	A02:2021	Critical	Environment variables + secrets manager (Vault)
SC-004	Weak JWT Implementation	A07:2021	Critical	Whitelist HS256 only, add expiry + jti claim
SC-005	IDOR	A01:2021	High	Add user_id to WHERE clause — ownership check
SC-006	Weak Password Storage	A02:2021	Critical	bcrypt/Argon2id — never MD5/SHA1 for passwords

15. AI Security Assistant

The AI Security Assistant integrates Groq's LLaMA 3 8B model to provide intelligent responses to security architecture questions. The assistant operates in two modes:

Live Mode: When a Groq API key is provided (free at console.groq.com), real LLaMA 3 responses are generated using the system prompt engineered for security architecture expertise.

Demo Mode: Without an API key, 8 intelligent pre-built responses cover all major topics including Zero Trust, STRIDE, OWASP, DREAD, microservices security, and pattern recommendations.

Quick action buttons allow one-click explanations of all 6 STRIDE categories, DREAD scoring, OWASP Top 10, and architecture pattern recommendations without typing.

16. User Authentication System

The framework implements a functional role-based user authentication system using SQLite for storage and SHA-256 with random salt for password hashing.

Feature	Implementation
Password Hashing	SHA-256 with 16-byte random salt (hex) — constant-time comparison via <code>hmac.compare_digest</code>
Storage	SQLite users table — no plaintext passwords ever stored
Roles	Three roles: admin, faculty, student — displayed in sidebar with color coding
Demo Accounts	admin/Admin@123, student1/Student@123, faculty1/Faculty@123 — seeded on first run
Registration	Username uniqueness enforced, minimum 8-char password, email validation
Session Management	Streamlit <code>session_state</code> — <code>logged_in</code> flag + <code>user_info</code> dict
Profile Page	Shows quiz history, analysis count, last login, role badge

17. Report Generation

Format	Generator	Contents	Use Case
HTML	html_generator.py	Cover, exec summary, components, STRIDE threats, risk, recommendations, mitigation	Browse results, presentation
PDF	pdf_generator.py	Same as HTML — paginated with ReportLab, dark theme	Academic submission, client delivery
JSON	report_generator.py	Complete machine-readable AnalysisResult.to_dict()	API integration, data exchange
CSV (Threats)	report_generator.py	ID, category, title, component, severity, risk, mitigation	Spreadsheet analysis
CSV (Recs)	report_generator.py	ID, priority, title, component, pattern, explanation	Remediation tracking

18. Sample Architecture Files

Filename	Format	System	Risk Level	Components
sample_ecommerce.json	JSON	E-Commerce Platform	High ~74%	13
sample_banking.yaml	YAML	Online Banking System	High ~85%	7
sample_hospital.xml	XML	Hospital Management	High ~76%	7
sample_microservices.puml	PlantUML	Microservices Application	High ~91%	13
sample_iot.json	JSON	IoT Monitoring System	High ~78%	7
secure_architecture_low_risk.json	JSON	Secure Banking App	Low ~0%	8
insecure_architecture_high_risk.json	JSON	Vulnerable Healthcare	High ~77%	8
xml_low_risk.xml	XML	Secure Government Portal	Low ~0%	7
xml_medium_risk.xml	XML	University Student Portal	High ~78%	8
xml_high_risk.xml	XML	Legacy Hospital System	High ~76%	9
xml_critical_risk.xml	XML	Vulnerable Banking System	High ~78%	11

19. Functional Requirements

FR No.	Requirement
FR-01	System shall accept architecture files in JSON, YAML, XML, and PlantUML formats
FR-02	System shall validate uploaded architecture files and report errors clearly
FR-03	System shall extract components, data flows, trust boundaries, and all attributes
FR-04	System shall apply 20+ STRIDE threat detection rules to every component and flow
FR-05	System shall calculate risk scores using Likelihood x Impact (1-25 scale)
FR-06	System shall classify risks as Low, Medium, High, or Critical
FR-07	System shall map each threat to corresponding OWASP Top 10 2021 categories
FR-08	System shall calculate DREAD scores for all detected threats
FR-09	System shall generate interactive attack tree visualizations
FR-10	System shall map threats to security design patterns from the 24-pattern library
FR-11	System shall generate prioritized, architecture-specific recommendations
FR-12	System shall display an interactive architecture graph with 5 view modes
FR-13	System shall provide a security controls heatmap per component
FR-14	System shall generate downloadable HTML, PDF, JSON, and CSV reports
FR-15	System shall save analysis results to a persistent SQLite database
FR-16	System shall allow users to reload, compare, and delete previous analyses
FR-17	System shall provide built-in sample architectures covering 4 input formats
FR-18	System shall provide an AI-powered security assistant with chat interface
FR-19	System shall implement role-based user authentication (admin/faculty/student)
FR-20	System shall display secure coding examples with vulnerable vs. fixed code

20. Non-Functional Requirements

NFR No.	Category	Requirement
NFR-01	Performance	Analysis of a 15-component architecture shall complete within 5 seconds
NFR-02	Usability	Interface must be usable by a developer without prior security expertise
NFR-03	Reliability	Application must not crash on invalid input — all errors handled gracefully
NFR-04	Security	Uploaded files validated; no code execution; parameterized SQL queries only
NFR-05	Portability	Must run on Windows, macOS, and Linux without modification
NFR-06	Deployability	Must deploy to Streamlit Community Cloud without additional configuration
NFR-07	Maintainability	Code must be modular, documented, and follow PEP 8 standards
NFR-08	Scalability	Modular architecture allows new STRIDE rules to be added without refactoring
NFR-09	Testability	All core modules must have corresponding pytest test cases
NFR-10	Accessibility	UI must be usable on standard screen resolutions (1280px+)

21. Test Cases

Test ID	Module	Test Case	Expected Result	Status
TC-01	JSON Parser	Parse valid E-Commerce JSON	13 components, 13 flows	Pass
TC-02	JSON Parser	Parse invalid JSON (syntax error)	ParseError raised	Pass
TC-03	JSON Parser	Parse JSON with missing optional fields	Defaults applied correctly	Pass
TC-04	YAML Parser	Parse Banking YAML file	7 components, 5 flows	Pass
TC-05	XML Parser	Parse Hospital XML file	7 components, 6 flows	Pass
TC-06	XML Parser	Parse malformed XML	ParseError raised	Pass
TC-07	PlantUML Parser	Parse Microservices PUML	13 components, 15 flows	Pass
TC-08	STRIDE Engine	Analyze E-Commerce — detect all 6 categories	All 6 STRIDE categories	Pass
TC-09	STRIDE Engine	Internet-facing no auth → Spoofing threat	SPO-001 triggered	Pass
TC-10	STRIDE Engine	Unencrypted flow → Tampering threat	TAM-F001 triggered	Pass
TC-11	STRIDE Engine	No logging → Repudiation threat	REP-001 triggered	Pass
TC-12	STRIDE Engine	No duplicate threats per component	Unique threats only	Pass
TC-13	Risk Calculator	Score 1-4 classifies as Low	Level = Low	Pass
TC-14	Risk Calculator	Score 17-25 classifies as Critical	Level = Critical	Pass
TC-15	Risk Calculator	Empty threats → 0% risk	overall_risk_pct = 0	Pass
TC-16	Risk Calculator	3x5 score → 100% risk	overall_risk_pct = 100	Pass
TC-17	Integration	Full pipeline: parse → analyze → report	Valid AnalysisResult	Pass
TC-18	HTML Report	Generate HTML for E-Commerce result	Valid HTML with threats	Pass
TC-19	PDF Report	Generate PDF for E-Commerce result	PDF magic bytes %PDF	Pass
TC-20	JSON Report	Serialize and deserialize AnalysisResult	Identical data roundtrip	Pass

22. Future Scope

Enhancement	Description
ML-Enhanced Detection	Train models on CVE databases for pattern-based threat prediction
MITRE ATT&CK Mapping	Map detected threats to MITRE ATT&CK tactics and techniques
Compliance Mapping	Map recommendations to GDPR, HIPAA, PCI-DSS, ISO 27001 controls
CI/CD Integration	GitHub Actions workflow to assess architecture on every commit
Custom Rule Engine	Allow organizations to define domain-specific threat detection rules
Architecture Comparison	Side-by-side comparison of secure vs. insecure architecture versions
Multi-User Collaboration	Shared assessment sessions for team-based architecture review
Export to Threat Dragon	Export results to OWASP Threat Dragon format for further analysis
IDE Plugin	VS Code extension for in-editor architecture security assessment
Real CVE Integration	Fetch live CVE data from NVD to enrich threat analysis
React/Next.js Frontend	Full rebuild with modern React UI, FastAPI backend, PostgreSQL
LLM Fine-Tuning	Fine-tune a security-specific LLM on architecture assessment data

23. Conclusion

The Security-Centric Architecture Assessment Framework successfully demonstrates that automated security analysis can and should begin at the architecture design stage. By providing 20+ STRIDE detection rules, DREAD risk quantification, OWASP Top 10 mapping, 12 architecture patterns, 24 security patterns, attack tree visualization, and professional report generation — all without requiring code or deployment — the framework fills a genuine gap in the software security toolchain.

The framework is practical, demonstrable, and extensible. It serves as both an academic research prototype and a foundation for production-ready architecture security tooling. The modular design allows new detection rules, patterns, and output formats to be added independently without affecting existing functionality.

Project Statistics Summary:

Metric	Value
Total Application Pages	17
STRIDE Detection Rules	20+
Architecture Patterns	12
Security Design Patterns	24
Secure Coding Examples	6
Quiz Questions	20
Research Resources	13 (NIST/OWASP/CIS/IEEE)
Sample Architecture Files	11 files, 4 formats
Input Formats Supported	4 (JSON, YAML, XML, PlantUML)
Report Output Formats	4 (HTML, PDF, JSON, CSV)
Lines of Code (approx.)	6,000+
Test Cases	20
Python Dependencies	14
Database	SQLite (zero-dependency)